

Enhancing IoT Network Security through Intelligent Device Identification Using Machine Learning

Tarek Azzabi
Laboratory LR-19-DN02
Tunisian Naval Academy
azzabitarek@gmail.com

Mohamed Hechmi Jeridi
Tunisian Naval Academy
Menzel Bourguiba 7050, Tunisia
mohamedhechmi.jeridi@enit.utm.tn

Mohamed Aziz Kacem
SYSCOM - LR-99-ES21
National Engineering School of Tunis – ENIT
azizkacem110@gmail.com

Tahar Ezzedine
SYSCOM - LR-99-ES21
National Engineering School of Tunis – ENIT
tahar.ezzedine@enit.utm.tn

Abstract—As the Internet of Things (IoT) continues to expand, so do the security vulnerabilities associated with its devices. This paper introduces IoTID Acad, a machine learning-based framework designed to enhance IoT network security through intelligent device identification. By analyzing network packet characteristics, IoTID Acad predicts the type or specific identity of IoT devices, with a focus on low-power, IPv6-based devices. The system is capable of detecting critical threats, including Denial-of-Service (DoS) attacks and battery-draining attacks. When integrated into an Intrusion Detection System (IDS), IoTID Acad effectively isolates malicious devices, significantly bolstering the security of IoT networks. This research underscores the potential of machine learning to address pressing security challenges in the rapidly evolving IoT ecosystem.

Index Terms—IoT security, Machine learning, Device identification, Intrusion detection system, IPv6, low power networks.

I. INTRODUCTION

IoT applications encompass a wide range of domains, including military, healthcare, agriculture, and more. As user demands evolve, new IoT applications emerge, driving the continuous growth of connected devices. According to Statista, global annual IoT revenue is projected to reach 336 billion dollars in 2024 and is expected to grow to 621.6 billion dollars by 2030 [1].

However, this rapid expansion has introduced vulnerabilities, such as malicious attacks, which pose significant security threats, especially at critical and governmental levels. These threats include Denial of Service (DoS) attacks and battery consumption attacks. Thus, it is imperative to effectively secure IoT systems against such attacks.

This work aims to design a system that enhances IoT network security using machine learning techniques for device identification. We propose the development of a classification model, named "IoTID_acad" capable of identifying devices based on their unique fingerprints. The model will be trained using a custom dataset simulated in the Cooja software and will detect specific attacks, such as DoS and battery consumption attacks, while ensuring compatibility with IPv6 devices.

Finally, an intelligent Intrusion Detection System (IDS) will be created, leveraging this model to identify malicious devices effectively.

II. RELATED WORKS

Currently, many studies focus on identifying IoT devices; however, their applicability to real-world IoT deployments remains limited. This is because these identification methods frequently rely on analysis at higher layers of the network stack (e.g., application layer), while IoT devices often utilize diverse and complex communication protocols at lower layers (e.g., physical and link layers).

In this context, the IoT Sentinel [2] used packet characteristics. The study used data from Aalto University to extract these features, enabling the use of a behavior-based fingerprint. A specific fingerprint was created for each device based on 23 features from the first 12 packets of each device, totaling a fingerprint with 276 values. It is important to note that these 12 packets do not constitute a full flow; rather, they are a sequence of packets from the same MAC address. In addition, features include information about the source IP address, destination address, protocol, and the used properties. However, since it merges packets from MAC addresses, this approach cannot identify non-IP devices. Additionally, the IP address counting function used in this study is unlikely to be generalizable beyond the Aalto dataset, as it refers to the number of devices with which each device communicates, which depends on the environment. In this study, 17 of the 27 device types were detected with identification accuracy greater than 95%, while 10 had an accuracy of around 50%.

Two other notable studies contributed to the creation of the Aalto dataset. The first study[3] extracted a fingerprint with 67 features from Ethernet, IP, UDP, and TCP packet headers over 20 to 21 consecutive packets. This study used network statistics in addition to features obtained from the packets. These statistics vary as devices move from one network to another and are characterized by their interrelationships, which makes generalization unlikely. This study classified devices

with an accuracy of 90.3% using various machine learning techniques.

On the other hand, the second study[4] achieved 95% accuracy using 33 features selected from 212 using a genetic algorithm (GA). However, this study has limitations in terms of generalization, as it uses features that cannot generalize network behavior, such as IP Identification (ID), TCP acknowledgment, TCP sequence, and source and destination port numbers. As a result, this study successfully classified only 23 of the 27 device classes.

Another study, IoTSense[5], used some characteristics from IoT Sentinel for its own evaluation. They used features extracted from 17 protocols that reflect IoT device behavior, adding other features such as payload and entropy. This list of features was used for each set of 5 packets from each device to produce a fingerprint. This study achieved 99% accuracy, but in terms of generalization, it lacked access to large datasets. The extraction methods uses device private information, which threaten the privacy of devices. To overcome these limitations, authors in [6] proposed a Linear Residual Neural Network-based extraction method, Res-DFNN, bu using network packet data in the pcap file.

Finally, a study named IoTDevID[7] addresses the transfer problem using a set of features extracted from the TCP/IP protocol architecture. This study achieved 98% accuracy with UNSW data. However, it has limitations related to the use of IPv4 addresses, as we have seen with IoT protocols. Although this method is suitable for non-IP environments, it is resource-intensive, which is significant in the context of a large-scale device network[8].

III. THEORETICAL MODEL AND PROPOSED APPROACH

A. Network Component

1) *Graph Theory*: Graph theory provides a framework for modeling complex relationships through nodes and edges. It aids in visualizing and understanding systems like communication networks, road networks, and electrical circuits, providing a versatile tool for analyzing connections.

2) *Relationship between DoDAGs and Graph Theory*: The RPL protocol supports multiple DoDAGs (Destination-Oriented Directed Acyclic Graphs) for efficient traffic routing using optimal metrics. Its topology forms a spanning tree where nodes are vertices, links represent traffic, and the root node defines the primary DoDAG. Each node selects successors from neighbors based on routing metrics, prioritizing the most optimal preferred parent.

B. Data Component

1) *Payload Entropy*: Payload entropy measures the information content in a data packet, reflecting the type and size of exchanged messages. Plain text typically has low entropy, while formats like XML, compressed, or encrypted data show higher entropy. For instance, temperature sensor data may exhibit different entropy levels than motion sensor

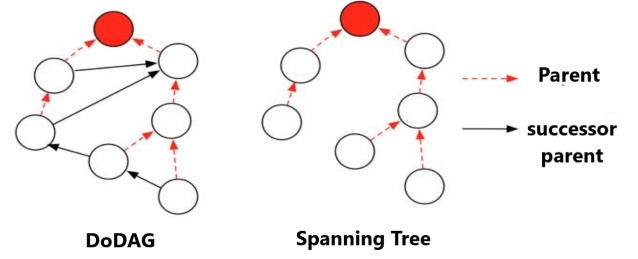


Fig. 1: DoDAG as compared to a spanning tree[9]

data. Shannon's famous formula for calculating entropy is given by:

$$H_m = \sum_{i=1}^{256} p_i \log_2(p_i) \quad (1)$$

where p_i represents the probability of occurrence of a byte in an exchanged message.

The inclusion of this feature helps improve the classifier's accuracy by providing more information about the type of messages exchanged between IoT devices[10].

2) *Payload Length*: This measure represents the payload size in a TCP/UDP message, distinguishing devices based on message size rather than packet length. The algorithm initializes an empty payload list, traverses packet files, checks for TCP, UDP, or ICMP protocols, and adds matching payloads to the list. The payload length is then calculated in bytes [10].

3) Data Augmentation: The ACTGAN Model

The ACTGAN model employs a structured approach to generating conditional tabular data. First, it applies mode-specific normalization to each continuous column C_i , using a variational Gaussian mixture model (VGM) to estimate the number of modes m_i and adjust a Gaussian mixture accordingly. Next, it uses a conditional generator and a sampling-based training method to address the imbalance in discrete columns, modeling the conditional distribution as $P(G|row_{D_i} = k)$. Finally, ACTGAN utilizes fully connected networks to capture all possible correlations between columns, combining recent techniques to train a high-quality model[10].

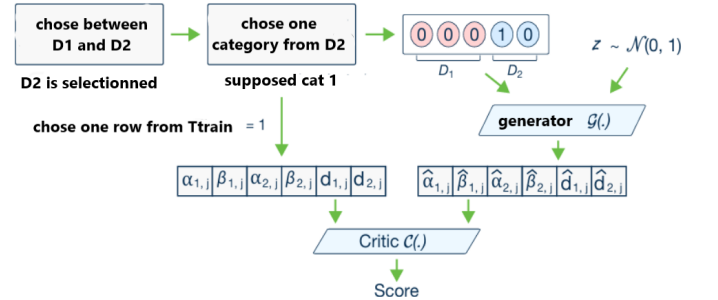


Fig. 2: ACTGAN model[10]

4) *Genetic Algorithm*: A genetic algorithm searches for the extrema of a function defined over a data space. The figure 3 depicts the workflow of genetic algorithm. The algorithm consists of five key steps as follows:

1) **A population element encoding scheme:** This step aims to structure the data and typically follows a mathematical modeling phase of the problem. The quality of the data encoding determines the success of the algorithm.

2) **A mechanism for generating the initial population:** This step involves producing a heterogeneous population of individuals capable of serving future generations. The choice of the initial population can either accelerate or slow down the convergence to the optimal solution.

3) **An objective function to optimize:** This function returns the evaluation or fitness value.

4) **Operators that diversify the population across generations and explore the state space:** The crossover operator recombines genes of existing individuals in the population, while the mutation operator ensures exploration of the state space.

5) **Sizing parameters:** This includes the total number of generations or the stopping criterion, population size, and the probabilities of applying crossover and mutation operators.

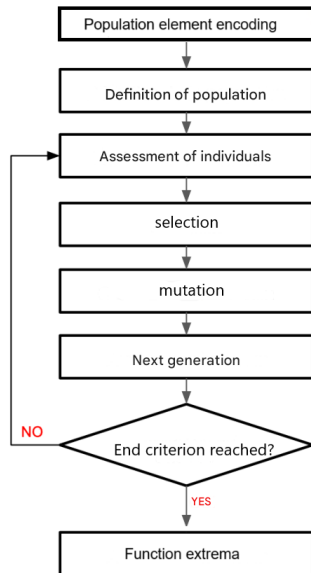


Fig. 3: Genetic approach phases

C. Learning Component

1) *Individual, Aggregated, and Mixed Packet Methods:* Merged packets aggregate multiple individual packets into a single transmission, such as grouping several sensor readings before sending them to a server. While merging improves efficiency, there are no standards for the size or duration of the merging process. To maintain speed and resource efficiency, our study focuses on individual packets for discrimination.

However, analyzing packets individually can lead to ambiguities in certain transfer failures. To address this, we use an aggregation method that groups packets by MAC address and ML-assigned labels, improving accuracy by ignoring mislabeled packets. Unlike previous approaches, our method analyzes behavior at the aggregation level rather than merging packets directly.

To handle limitations caused by MAC address transfer issues, we implemented a mixed method that uses exceptions for better reliability.

D. Approach

The adopted approach secures IoT networks by identifying devices based on their behavior, even in the presence of data transfer issues. Leveraging machine learning, it utilizes precise fingerprints to identify devices, including those using non-IP protocols or operating on low power. To address IPv4 exhaustion and resource constraints, IPv6 adaptation is integrated via 6LoWPAN, enabling scalable and energy-efficient communication. By employing the IETF architecture, the model extracts distinctive features from protocol headers, providing a robust solution for IoT device identification and security.

E. Simulation and Data Processing

The used method involves developing an IoT network using the IETF architecture in simulators like Cooja. Network traffic was captured through Cooja, generating pcap files for feature extraction, and attack scenarios were simulated to create malicious signatures for identifying attack types. To address limited dataset size, synthetic data was generated, and packet analysis using Wireshark extracted 55 features. Data refinement included session management, duplicate removal, and metric calculations like payload length and entropy. Feature selection involved scikit-learn voting methods, cross-validation, and genetic algorithms, creating a robust dataset for model generalization.

F. Machine Learning Model Optimization

Six machine learning models—Decision Tree (DT), Random Forest (RF), Naïve Bayes (NB), K-Nearest Neighbors (KNN), Support Vector Machine (SVM), and Gradient Boosting (GB)—were trained with hyperparameter tuning to identify the most effective algorithm based on its F1 score. This approach aims to optimize the parameters of each model to prevent overfitting and underfitting.

G. Attack Approach

To simulate real-world attack scenarios on IoT nodes, we developed two malicious codes in C under Cooja: one for a Denial of Service (DoS) attack and the other for a battery consumption attack. In this section, we explain the methodology behind each attack.

1) *DoS Attack (Denial of Service):* As shown in Figure 4, we created malware to flood the server with a high volume of requests. Injected into a node, it established a connection and send a large number of successive requests faster than the server's response time, ultimately overwhelming it and causing a denial of service.

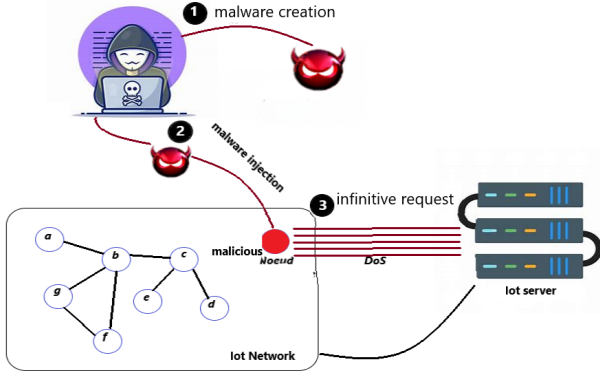


Fig. 4: DoS attack

2) *Battery Consumption Attack*: As illustrated in Figure 5, we developed malware that increases the battery consumption of target nodes. The code was injected into a node and integrated into a sensor network. The malicious node established connections with neighboring nodes and sent repeated unwanted messages, causing increased battery consumption. This process continued as each connected node became affected.

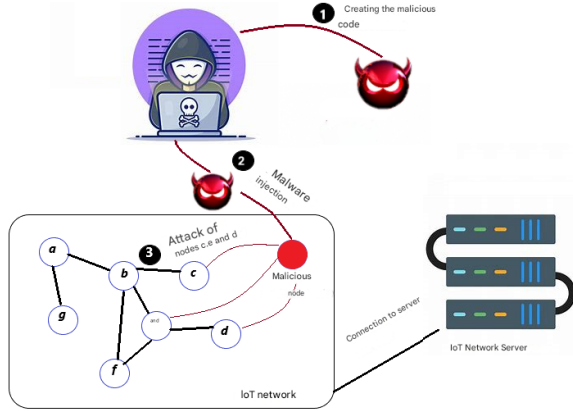


Fig. 5: battery consumption attack

IV. RESULTS AND ANALYSIS

A. Case study: Fixed IoT network

In this case study, we addressed the scenario where the IoT network is fixed, meaning the sensors do not change their positions relative to each other. In this context, we created a network of 11 sensors using Cooja, as shown in Figure 6a, and conducted the simulation for 10 sessions. For each session, we captured the data in pcap files. We simulated several attack scenarios, including a Denial of Service (DoS) attack, where Node₁₂ targeted Node₁ (the server), as illustrated in Figure 6a. Similarly, the battery consumption attack was demonstrated in Figure 6b, where Node₁₂ attacked different nodes, increasing their battery consumption.

B. Feature Extraction

This section outlines the feature extraction process. We identified relevant features from each layer of the IETF

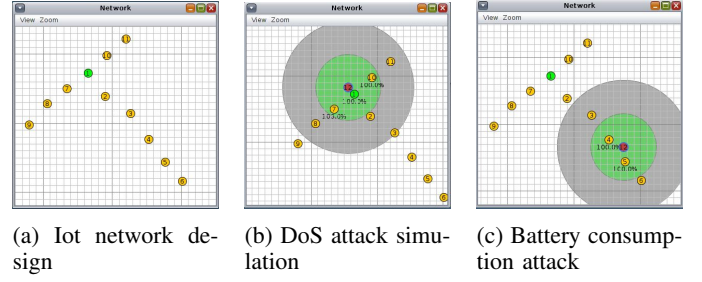


Fig. 6: Fixed IoT network

architecture (Figure 7). A Python script was developed to analyze filtered pcap files. This script included functions for port classification, entropy calculation, and payload analysis. Features were extracted from each packet, with MAC addresses mapped to device names and protocol information appended. Attack fingerprints were generated and integrated with the initial dataset. After removing duplicate entries, the dataset was divided into training 80%, validation 10%, and testing 10% sets.

IETF IoT Protocol Stack		
CoAP	Application Layer	DNS_id, DNS_gr, DNS_opcode, DNS_aa, DNS_tc, DNS_rd, DNS_ra, DNS_s, DNS_of, DNS_of, DNS_rode, DNS_qdcount, DNS_answercount, DNS_rsourcecount, DNS_answercount, ICMPv6_type, DHCP_options, ARP_header, LL_C, drap, EAPOL_version, BOOTP_op
UDP	Transport Layer	UDP_sport, UDP_dport, UDP_len, TCP_sport, TCP_dport
IPv6	Network Layer	IP_version, IP_tc, IP_fl, IP_plen, IP_hlim
6LoWPAN	Adaption Layer	_reserved, tf, type
IEEE802.15.4 MAC	Data-link Layer	fcf_reserved, Seqnum, fcf_panidcompress, fcf_ackreq, fcf_pending, fcf_security, fcf_arcdemomode, fcf_framever, fcf_desaddrmode, fcf_reserved_2, Mac,
IEEE802.15.4 PHY	Physical Layer	

Fig. 7: Features for each layer of the IETF

C. Dataset creation

The first result we obtained was an initial dataset with 20,600 rows, stored in a CSV file, consisting of 55 features and 12 classes. The two types of attacks were grouped into the same class. We then used this dataset to train the ACTGAN model and generated synthetic data consisting of 400,000 rows over a duration of 5 hours. To evaluate the generated synthetic data, we compared it with the initial dataset.

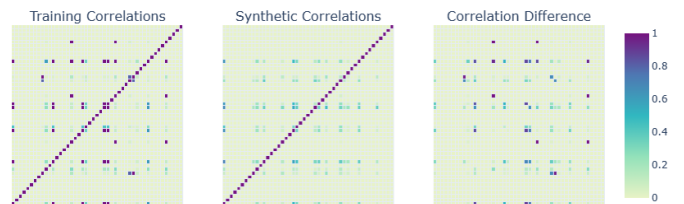


Fig. 8: Results of Initial and Synthetic Dataset Creation

As shown in Figure 8, the difference between the two

TABLE I: Results of individual packet classification models

ML_alg	Accuracy	b_acc	Pre	Rec	F1	tra_T	test_T
DT	0.80	0.81	0.86	0.81	0.8277	2.9	0.07
RF	0.82	0.81	0.86	0.81	0.8276	156.83	28.31
NB	0.16	0.16	0.23	0.16	0.1194	2.78	0.15
KNN	0.71	0.71	0.75	0.718	0.68	20.19	20.2
SVM	0.68	0.63	0.7	0.63	0.6493	64.87	166.04
GB	0.7	0.69	0.79	0.69	0.7241	923.34	8.34

datasets, represented in the correlation matrix, is minimal, indicating a 93% similarity between them.

D. Features vote result

Figure 9 indicates that, out of the 55 features, 13 did not receive any votes across the six voting methods. The 13 features are: IP_fl, DNS_nscount, DNS_qdcount, DNS_rcode, DNS_tc, TCP_sport, TCP_dport, BOOTP_op, LLC_dsap, IP_tc, DNS_z, ARP_hwtype. These features will be eliminated subsequently.

E. Cross-validation result

The results vary significantly depending on the model evaluation method used, whether cross-validation or the separation of training and testing sets, as shown in Figure 10. This variation occurs because cross-validation often samples training and testing data from the same session, making it easier to generalize session-specific models.

To address this, we chose to use our initial data instead of cross-validated data. This decision aligns with our focus on session attributes such as IP version, EAPoL version, and seqnum, which are randomly assigned per session and limit generalization. While these attributes initially showed promising results, they proved too session-specific to generalize the model effectively. Additionally, features like UDP sport, UDP dport, sport23, dport23, and dport bare contributed to overfitting, further hindering generalization. As a result, we will eliminate all these features moving forward.

F. Genetic algorithm results

To determine the most relevant features for further analysis, we employed a genetic algorithm to select features based on the voting method, while eliminating those associated with specific sessions or ports. The initial generation, consisting of 33 features, achieved a precision of 0.82, representing the highest score across all generations compared.

1) *Performance Results of Packet Classification Models:* Depending on the selected hyperparameters and the evaluation of each classification model, we obtained the results presented in Table I. The analysis revealed that DT and RF models achieved the highest accuracy while ensuring rapid execution times. Conversely, the NB model, though less accurate, demonstrated the quickest execution. The KNN, SVM, and GB models delivered good accuracy but required longer execution times. Since IoT device detection must be performed in real-time with high accuracy, we opted for the DT model, which offers the fastest test time while maintaining high accuracy.

2) *Performance results of aggregation group size:* The F1-score performance of the DT model based on the aggregation group sizes is illustrated in Figure 11. We noticed that a group size of 1 yields the best F1-score compared to other group sizes. For this reason, we chose this group size for the continuation of our study.

3) *Results of classification for individual, mixed, and aggregated packets:* The results obtained by the proposed model are summarized in Table II. This model achieved an accuracy of 82%, with an F1-score of 82.77%, demonstrating the high performance of our model. Additionally, we observe that the accuracy of this model increases by 2% when comparing the results of individual packets with those of aggregated packets, highlighting the effectiveness of our method to improve accuracy.

TABLE II: Results of classification for individual, mixed, and aggregated packets

Classification	Accuracy	F1 score	test_T
Individual	0.80	0.80	0.01
Aggregated	0.82	0.8277	0.05
Mixed	0.82	0.8277	0.01

4) *Results of classification by the mixed method for each class:* The model's performances are listed in the table III. For the malicious_devices class, the results show that the model achieves an accuracy of 90%, indicating a low false positive rate. Additionally, a recall of 83% suggests a low false negative rate. For classes 11 and 6, we observed a small number of false negatives.

TABLE III: Results of Classification for Individual, Mixed, and Aggregated Packets

Node	Accuracy	Recall	F1 score
Malicious_devices	0.8952	0.828	0.800.86047
Mote1	0.725854	0.90	0.8051
Mote2	0.89366	0.80	0.836952
Mote3	0.98952	0.775522	0.836952
Mote4	0.640989	0.905	0.7507
Mote5	0.845732	0.819658	0.832491
Mote6	0.904036	0.818388	0.859082
Mote7	0.947789	0.760365	0.843794
Mote8	0.8842881	0.783905	0.831073
Mote9	0.69115	0.812340	0.745671
Mote10	0.981285	0.764393	0.859365
Mote11	0.975354	0.768198	0.859470

V. CONCLUSION

This work introduces a new approach to secure IoT devices by developing a supervised machine learning model that identifies and classifies devices based on network behavior and digital fingerprints. The study involved simulating an IoT network using the IETF architecture with 6LoWPAN adaptation, followed by feature selection through session management, feature extraction, and synthetic data generation. The model, enhanced by voting mechanisms and genetic algorithms, was integrated into an Intrusion Detection System (IDS) to detect attacks like DoS and battery consumption, improving IoT network security.

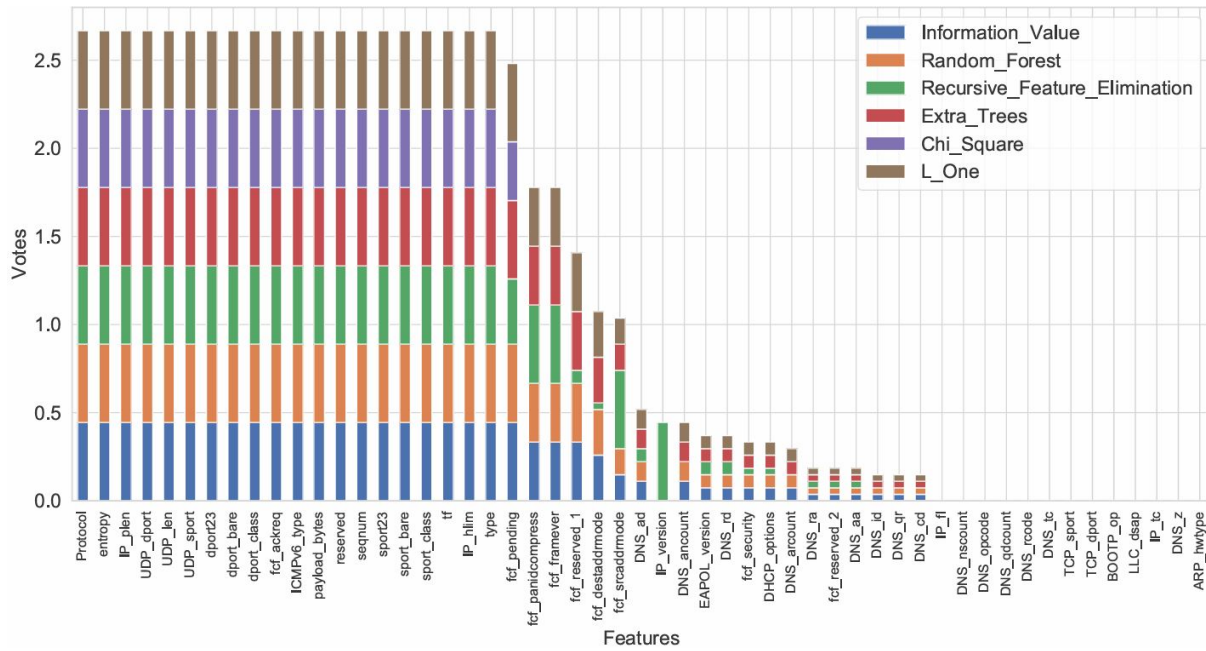


Fig. 9: Features vote result



Fig. 10: Comparison of Cross-Validation and Isolated Validation Methods

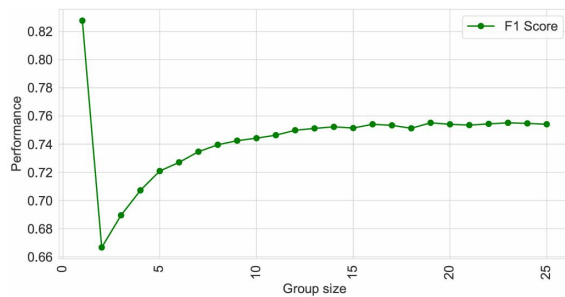


Fig. 11: Results of Individual Packet Classification Models

This research demonstrates the critical role of machine learning in mitigating IoT security threats, with future plans to develop an IoT security platform for custom models, a Security Information and Event Management (SIEM) system, and Digital Twins IDS. These innovations will enable real-time monitoring, predictive threat analysis, and improved device identification, further strengthening IoT security.

REFERENCES

- [1] Statista, "IoT total revenue worldwide 2019-2030," Statista, 2024. <https://www.statista.com/statistics/1194709/iot-revenue-worldwide/>
- [2] Miettinen, M., Marchal, S., Hafeez, I., Asokan, N., Sadeghi, A.R. and Tarkoma, S., 2017, June. Iot sentinel: Automated device-type identification for security enforcement in iot. In 2017 IEEE 37th international conference on distributed computing systems (ICDCS) (pp. 2177-2184). IEEE.
- [3] Hamad, S.A., Zhang, W.E., Sheng, Q.Z. and Nepal, S., 2019, August. Iot device identification via network-flow based fingerprinting and learning. In 2019 18th IEEE international conference on trust, security and privacy in computing and communications/13th IEEE international conference on big data science and engineering (TrustCom/BigDataSE) (pp. 103-111). IEEE.
- [4] Aksoy, A. and Gunes, M.H., 2019, May. Automated iot device identification using network traffic. In ICC 2019-2019 IEEE international conference on communications (ICC) (pp. 1-7). IEEE.
- [5] Bruhadeshwar, B., Maalvika, B., Jordan, P., Hossein, S., Indrakshi, R., Indrajit, R., 2018 IoTSense: Behavioral Fingerprinting of IoT Devices <https://doi.org/10.48550/arXiv.1804.03852>
- [6] Zhong, Y. and MChen, Y. W(2024)Res-DFNN: An NN-Based Device Fingerprint Extraction Method Using Network Packet DataSymmetry10.3390/sym160404316:4(443) <https://doi.org/10.3390/sym16040443>
- [7] Kostas, K., Just, M. and Lones, M.A., 2022. IoTDevID: A behavior-based device identification method for the IoT. IEEE Internet of Things Journal, 9(23), pp.23741-23749.
- [8] Mariam, M. M., Wee, L., T., Mohammad A. Ali Z., "A Comparative Analysis of IoT Device Fingerprinting Methods", 2024 34th International Telecommunication Networks and Applications Conference (ITNAC), pp.1-7, 2024.
- [9] Boubekeur, F., 2016. Les arbres couvrants de la théorie à la pratique. Algorithmes auto-stabilisants et réseaux de capteurs (Doctoral dissertation, Université Pierre et Marie Curie-Paris VI).
- [10] Xu L, Skoularidou M, Cuesta-Infante A, Veeramachaneni K (2019) Modeling tabular data using conditional GAN. Adv Neural Inf Process Syst 32, no. NeurIPS, 2019. <https://arxiv.org/abs/1907.00503>